Progress report

Plate-specific recognizer

20 October 2026

Code: `https://github.com/S1norin/inno-detecting-license-plates`

Contributors

Name	Email
Mikhail Krylov	m.krylov@innopolis.university
Maximilian Mifsud Bonici	m.mifsudbonici@innopolis.university
Andrei Polevoi	a.polevoi@innopolis.university
Evgenii Khovalyg	e.khovalyg@innopolis.university

1 New approach

We trained CRNN recognizer for license plates. Convolutional layers read the plate image, two bidirectional LSTM layers track the sequence of characters, and a CTC output turns the sequence into text. It replaces EasyOCR from week 1.

- What it is: a CRNN + BiLSTM + CTC text recognizer, about 3 million parameters
- What it replaces: the EasyOCR recognition stage
- The code is in `models/crnn/`

2 Motivation

In week 1 the detector was strong, but the recognizer was weak. EasyOCR read only 35% of characters correctly and got 0 out of 800 whole plates right on the test set. The problem was that a general-purpose OCR model simply does not transfer well to plate crops.

So we trained a model specifically on plates. We expected character accuracy to rise well above 0.35 and exact-plate accuracy to rise from 0.

3 Implementation

Aspect	First approach	Second approach
Model	EasyOCR 1.7.2, pretrained	CRNN: 4 conv blocks, 2 BiLSTM layers, CTC head
Training data	none (used as-is)	3,500 original-scale train crops from CCPD
Training	none	50 epochs, Adam lr 0.001, batch 32, seed 42, CPU only, ∼38 min
Character set	built-in, general text	the 58 characters from the training plates
Input	engine-specific	one grayscale crop, 48 × 256, aspect ratio kept
Evaluation	same harness, two protocols	unchanged

Table 1: Main differences from the first approach.

4 Initial results

Test split, both engines through the same harness:

Protocol / setting	Baseline (EasyOCR)	Second approach (CRNN)
ocr_only — char. accuracy	0.3474	0.7538
ocr_only — exact plates	0 / 800	269 / 800
end_to_end — char. accuracy	0.3240	0.6561
end_to_end — exact plates	0 / 800	133 / 800

Table 2: Test results, second approach vs baseline.

- The expected improvement happened: character accuracy more than doubled (34.7% → 75.4% on clean crops, 32.4% → 65.6% on the full pipeline), and exact plates went from 0 to 269.
- The full pipeline is still below the clean-crop number (65.6% vs 75.4%): the detector misses 17 test plates, and a detector box is a rougher crop than the ground-truth box.
- Large plates are read much worse than small ones (57% vs 86%), because we squash them into a 48 px tall input; the 10–19 degree tilt bin is the weakest (51%, only 63 samples).

Remaining work

- Fine-tune YOLO11 on our training split and compare it to the pretrained detector, then re-run end-to-end recognition with the better detector.
- Try a taller input for large plates to improve their accuracy.
- Final submission (10 November): failure-case analysis, runtime and memory numbers, reproducibility check on a clean clone, two-page summary, and demo.

References

- CRNN details and reproduction: `models/crnn/REPORT.md`

- Recognition baseline: `baseline/ocr/README.md`
- Detection baseline: `baseline/detection/README.md`
- Repository: `https://github.com/S1norin/inno-detecting-license-plates`